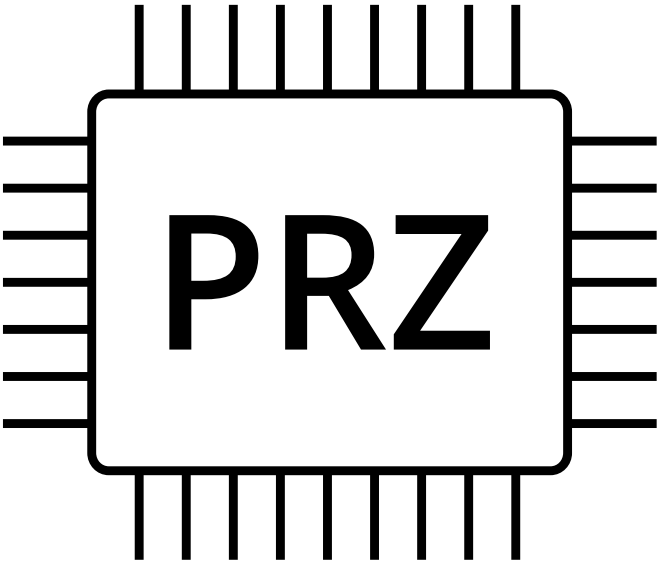
TECHNISCHE UNIVERSITÄT DRESDEN

Chair of Highly-Parallel VLSI Systems and
Neuro-Microelectronics

Processor Design Lab

Kreacher RISC-V SoC Task Description



For participants of the course only.
August 21, 2025

Contents

1 General Description **1**

2 RISC-V SoC Architecture **3**
2.1 Architecture Overview 3
2.2 Pin Description 3
2.3 Instruction Set Architecture 5
2.4 Memory Structure 6
2.5 Interrupts 7
2.6 Reset and Handler Addresses 8
2.7 SPI Interface 8
2.8 Program Memory Initialisation 11

3 Software **13**
3.1 Directory layout 13
3.2 Core Library 13
3.3 Testcase Software 15

4 Task Extension for Larger Groups **17**
4.1 Architecture 17
4.2 Interrupt Controller 17
4.3 Software 19

Bibliography **21**

List of Figures

2.1 Kreacher SoC Architecture 4
2.2 Interrupt request and acknowledge sequence 8
2.3 SPI single access, general. 10
2.4 SPI single read access 10
2.5 SPI single write access 10
2.6 SPI burst access, general. 10

4.1 Kreacher Extended SoC Architecture 18
4.2 Interrupt request and acknowledge sequence in mode 1 19

List of Tables

2.1 External Interface 4
2.2 Privileged instructions 5
2.3 CSRs 5
2.4 Signed division 6
2.5 Memory structure visible to the core 7
2.6 Memory structure visible to the init-controller 7
2.7 Reset and handler addresses 8

4.1 Register map of IRQ controller 18
4.2 Modified reset and handler addresses 20

Acronyms

CSR	control and status register
ISA	instruction set architecture
LSB	least significant bit
MSB	most significant bit
SoC	system-on-chip

1 General Description

'Any intelligent fool can make things bigger, more complex, and more violent. It takes a touch of genius—and a lot of courage to move in the opposite direction.'

(E. F. Schumacher)

The HPSN RISC-V microcontroller (*Kreacher*) is a RISC-V ISA based processor integrated into a system-on-chip (SoC) and aimed at general processing tasks. The basic features of *Kreacher* are:

- RISC-V instruction set architecture (ISA),
- 64 bit, little-endian data path,
- fully synchronous single clock architecture,
- 32 kB built-in program memory (PRAM),
- 2 general purpose interrupts,
- SPI bus interface for peripheral device and/or external memory attachment,
- PRAM initialization after reset,
- GCC support for higher level software development.

The architecture is described in Chapter 2. The provided compiler setup is explained in Chapter 3.

Depending on project group sizes, the tasks to be implemented are:

- Groups of 3 and less: reduced instruction set without *C* (compressed) extension.
- Groups of 4: full task without extension in Chapter 4.
- Groups of 5: full task with extension in Chapter 4.

2 RISC-V SoC Architecture

'Well, you sound vewy sure. Have you checked?'

(Monty Python, *Pontius Pilate in Life of Brian*)

2.1 Architecture Overview

Figure 2.1 gives an overview of the *Kreacher* SoC architecture. The chip will contain

- a processor core implementing the RISC-V ISA (see Section 2.3),
- 2 interrupt inputs (see Section 2.5),
- 32 kB on-chip SRAM, used as program memory,
- an external SPI interface (as described in Section 2.7),
- an init-controller for program memory initialization (see Section 2.8),
- a memory-controller giving the processor core and init-controller access to on-chip memory and the external bus interface according to the memory layout described in Section 2.4.

The external pin interface is described in Section 2.2.

2.2 Pin Description

The chip has external pins for clock and reset used for all components, the external SPI interface und interrupts. The pins are listed in table 2.1.

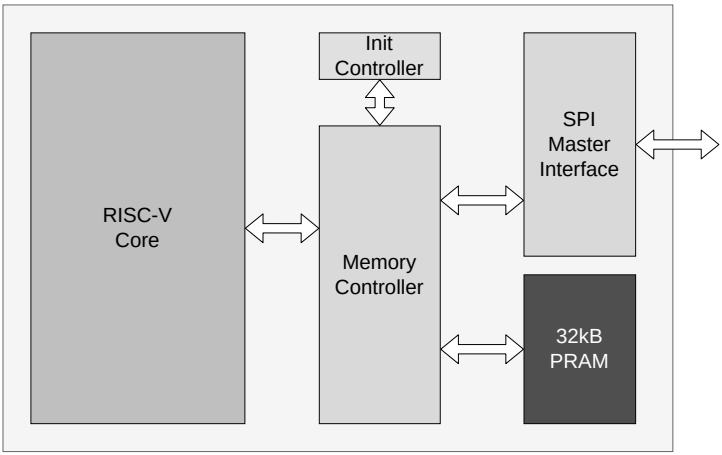


Figure 2.1: Kreacher SoC Architecture

Table 2.1: External Interface

Name	Width	Direction	Comments
`I_CLK`	1	In	System clock, posedge of clk is active
`I_A_RESET_L`	1	In	System reset, asynchronous, active-low
`O_SS`	1	Out	SPI-IF *SS*
`O_MOSI`	1	Out	SPI-IF *MOSI*
`O_MISO`	1	In	SPI-IF *MISO*
`I_INTR_H`	2	In	External Interrupt request
`O_INTR_ACK`	2	Out	External Interrupt acknowledge

Table 2.2: Privileged instructions to be supported

Instruction	Comment
MRET	return from (machine mode) interrupt.
WFI	wait for interrupt.

Table 2.3: CSRs to be supported

CSR	Access	Comment
`mstatus`	RW	machine status register for interrupt-enable.
`mepc`	RW	machine exception program counter.
`mcause`	RW	machine cause register.
`misa`	R	machine ISA register.
`mtvec`	R	machine trap-vector base-address, constant `0x01`.

2.3 Instruction Set Architecture

The processor core should implement the *RV64IMC* ISA[1] with *Zicsr* instructions. Reduced design groups may omit the compressed (*C*) instructions.

2.3.1 Privileged Instructions

Furthermore, some instructions (see table 2.2) and machine control and status registers (CSRs) (see table 2.3) of the privileged ISA[2] should be implemented to be standard conformal and support interrupts (*traps*). Code always runs in machine mode, no further privilege modes are supported. In general CSR registers are accessed by *Zicsr* instructions.

2.3.2 Signed Division

The current state of the specification ([1]) is unclear about what result is to be expected when calculating signed integer division results and remainders.

Table 2.4: Example: Signed division and remainder calculation

a	b	a / b	a % b
5	2	2	1
5	-2	-2	1
-5	2	-2	-1
-5	-2	2	-1

In this task the signed division should behave so that the absolute value of the results should be the same values as the results calculated by performing the unsigned operation with the absolute values of the arguments (see example in table 2.4).

2.4 Memory Structure

The SoC should contain 32 kB of SRAM for program memory. It is recommended to use 8 SRAM macros of type `HM_1P_GF28SLP_1024x32_1cr` with a size of 1 k×32 Bits each. Although intended to be used as program memory, write access by the core is supported.

The core will have a 64 bit address output. Addresses from the core are mapped as described in table 2.5.

As the 64 bit address values of the RISC-V ISA cannot be mapped to the 18 address bits used in the SoC, the topmost 46 bits of addresses are ignored while bit 17 set forces an external bus access. Ignoring the topmost 46 bits leads to addresses wrapping around, so address `0x40000` becomes address `0x0000` on the internal PRAM, effectively.

Forcing an external access can be used to map periphery into the external lowest 32 kB address range. In this case the periphery must be aware of the init-controller loading the program memory after reset.

The init-controller can explicitly select between on-chip SRAM and external interface in order to load an external program into the on-chip SRAM. It's memory map is shown it table 2.6.

Table 2.5: Memory structure visible to the core

address-range	target	mapped range
0x00000–0x07fff	on-chip SRAM	0x0000–0x07fff
0x08000–0x1ffff	external bus	0x8000–0x1ffff
0x20000–0x3ffff	external bus	0x0000–0x1ffff

Table 2.6: Memory structure visible to the init-controller

target-select	address-range	target
PRAM	0x0000–0x7fff	on-chip SRAM
external bus	0x0000–0x7fff	external bus

2.5 Interrupts

The SoC will have 2 interrupt inputs, *IRQ0* and *IRQ1*. Active interrupts will be handled at the end of an instruction. If both interrupts are active after an instruction, *IRQ0* has higher priority. The `MIE` bit of the `mstatus` is used for interrupt enabling, interrupts cannot be enabled independently.

Only vector interrupt mode needs to be supported.

Each interrupt is acknowledged separately using the corresponding bit of output bus *IRQ_ACK*. It shows that the interrupt request is accepted. When an interrupt is requested during an active command, this command needs to be finished before handling the interrupt. *IRQ_ACK* is valid for one clock cycle, when the interrupt handler address is loaded into the PC. Input signal *IRQ* stays high until the interrupt is acknowledged. See figure 2.2 for signal waveforms.

Note: A running interrupt handler cannot be interrupted by another interrupt, unless interrupts are again explicitly enabled by the handler (see [2]).

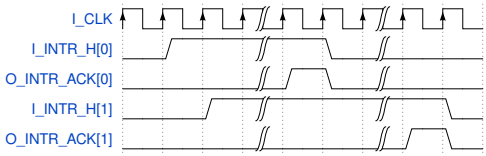


Figure 2.2: Interrupt request and acknowledge sequence

Table 2.7: Reset and handler addresses

handler	address
IRQ0	`0x0000`
IRQ1	`0x0004`
Reset	`0x0008`
Illegal instruction	`0x000c`

2.6 Reset and Handler Addresses

On reset, execution starts at address `0x0008`. Addresses for interrupt handlers start at `0x0000`. All handler addresses are listed in table 2.7.

Misaligned instruction read exception may be ignored (instructions are assumed to be aligend according to selected instruction set).

Accessing not implemented CSRs and writing to read-only CSRs count as illegal instructions.

2.7 SPI Interface

The SPI interface is an SPI master using a protocol for external memory and memory-like accesses via 17 bit addresses and 64 bit data words. Addresses are byte addresses, but accesses are always aligned to a whole 64 bit word. The protocol omits the 3 lowest address bits.

A transfer is initiated via 16 bits sent from master to slave, starting with 1 bit indicating whether the access is reading (0) or writing (1). The next 14

bits are external address bits 16 to 3 followed by 1 bit with constant 0 value giving 1 extra cycle for the slave to perform read accesses.

Depending on the access type, the following 64 cycles transfer read or write data word.

In case of a single word access, the access terminates after the data word is transferred. For a burst access the data word is immediately followed by the next data word (byte address is implicitly incremented by 8). After the last data word of a burst transfer the access is terminated.

2.7.1 Signals

Clock

Clock is shared with the SoC (pin `I_CLK`). Sampling occurs on positive clock edges.

SS

SPI slave select (pin `O_SS`) is a low-active signal indicating an SPI transfer to the external slave. SS gets low for the first cycle and stays low for the whole duration of the access. It becomes high again after the last access cycle.

MOSI and MISO

SPI master-out slave-in (MOSI, pin `O_MOSI`) is the data signal from master to slave. Master-in and slave-out (MISO, pin `I_MISO`) for data from slave to master. Data is transferred from most significant bit (MSB) to least significant bit (LSB).

2.7.2 Example Waveforms

Figures 2.3 and 2.6 show the generalized external SPI access, figures 2.4 and 2.5 show a single read and write.

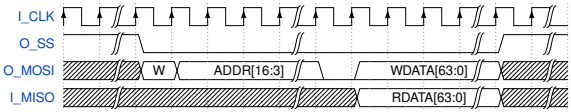


Figure 2.3: SPI single access, general.

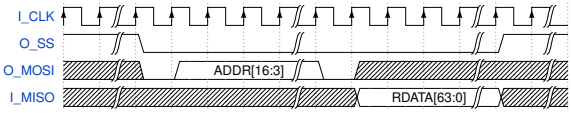


Figure 2.4: SPI single read access

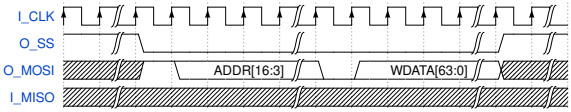


Figure 2.5: SPI single write access

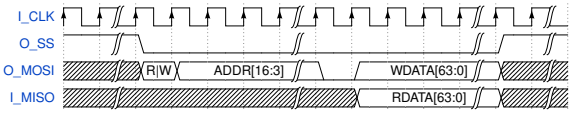


Figure 2.6: SPI burst access, general.

Single Transfers

A single read transfer (figure 2.3) starts with *SS* going to 0. At the same cycle a first bit indicating a write (1) or read (0) is transferred via *MOSI*.

The next 14 cycles transmit address bits 16 to 3 MSB first, followed by a single 0 bit to leave enough time for the slave to perform a read access in case of a read transaction.

In case of a read access (figure 2.4), the next 64 cycles transfer the read word via *MISO*, in case of a write access (figure 2.5), the next 64 cycles transfer the write data word via *MOSI*. The data word is also transferred MSB first.

The transfer is terminated by a cycle with *SS* going to 1 again.

Burst Transfers

For a long sequence of data to be transferred a burst mode can be implemented. The transfer is initiated similar to a single transfer until transmission of the data word with the extra bit after address transmission being set to 1 (figure 2.6).

As long as *SS* stays low, continuous data words will be transmitted via *MISO* (in case of read accesses) or *MOSI* (in case of write accesses). The address will be automatically incremented by one word (8 bytes) internally on master and slave sides without being retransmitted. The type of access (read or write) remains the same for all words of a burst.

The burst is terminated with *SS* going high after the last data word of the transfer.

2.8 Program Memory Initialisation

After reset, before starting execution at the reset-handler address, the internal program memory is initialized by the SoC via the external SPI interface.

Memory content from addresses `0x0000` to `0x7fff` is read into the internal memory. The initialization finishes on the last address or alternatively if *IRQ0* is activated. Initialization should make use of burst reads.

3 Software

'Beware of bugs in the above code; I have only proved it correct, not tried it.'

(Donald Knuth)

3.1 Directory layout

The *software* directory (located at `$ICPRO_DIR`) has the following layout:

```
software
├── corelib
│   ├── include
│   ├── lnk
│   └── source
└── tcsoftware
    ├── riscv_core.tc_sanity_asm
    ├── riscv_core.tc_<name>_asm
    ├── ...
    ├── riscv_core.tc_sanity
    ├── riscv_core.tc_<name>
    └── ...
```

The *corelib* directory encapsulates common parts of all testcases such as *linker script* and default *interrupt handlers* and related functions as well.

The *tcsoftware* directory collects all testcases.

3.2 Core Library

3.2.1 Linker script

The linker script can be found at `software/corelib/lnk/core.ld`.

To ensure that the reset vector is located at the begin of the instruction memory, the `ENTRY` statement is used. The layout is described in Section 2.6. In order to conform to this specification, a piece of assembler code[1] is used.

The memory layout is defined in the `MEMORY` section, which looks like this:

```
MEMORY
{
    ITCM (rx):  ORIGIN = 0x00000000, LENGTH = 16K
    DTCM (rwx): ORIGIN = 0x00004000, LENGTH = 16K
}
```

In this example, the instruction memory starts at address `0x00000000` and has a size of 16 kB. The data memory starts immediately afterwards.

The first section of the data memory is reserved for the *exitcode* of a program. It is initialized with the value `0xdeaddead` and will by overwritten with the return value of the `main` function or the value handed to the `exit()` function. This might be useful for self-checking testbenches, which simply need to evaluate memory address `0x4000`.

3.2.2 Interrupt Handlers

The following interrupt handlers can be overridden by the user by defining them in the C-code. However, to avoid unexpected behaviour, the `Reset_Handler` should only be overridden in special circumstances. The use of the default function (`__Reset_Handler`[2]), which initializes data memory and jumps to the `main()` function, is recommended.

```
void Reset_Handler (void);
void Illegal_Instruction_Handler (void);

void IRQ_00_Handler (void);
void IRQ_01_Handler (void);
```

When defining the handler function, make sure to use the compiler's `interrupt` attribute:

[1] located at `software/corelib/source/vectors.s`
[2] located at `software/corelib/source/handlers.c`

```
void __attribute__((interrupt)) IRQ_00_Handler (void)
{
    /* actual code... */
}
```

3.2.3 Return Value

As stated before, the return value (exitcode) of the program will by located at the `exit_code` section, which is translated by the initial linker script to `0x4000`.

3.3 Testcase Software

3.3.1 Creating Testcases using Assembler

In order to create raw assembler testcases (for validating a simple instruction sequence), the `riscv_core.tc_sanity_asm` can be used as a template and copied. The naming convention for the directory should follow:

```
<unitname>.<testcasedirectory>
```

e. g.

```
> cp -r riscv_core.tc_sanity_asm <unitname>.<tcname>
```

or in order to prepare the new testcase for svn commit, it is useful to do this with

```
> svn export riscv_core.tc_sanity_asm <unitname>.<tcname>
> svn add <unitname>.<tcname>
```

to avoid copying unversioned files.

After editing the assembler source code, the file can be translated to a *verilog* memory image by running:

```
> make
```

If the naming convention as mention above is met, the

```
> make export
```

command will copy the *verilog* memory image (`app.mem`) to:
`$ICPRO_DIR/units/<unitname>/simulation/ncsim/<tcname>`.

However, it is also possible to invoke the simulation directly by running the

```
> make rungui
```

target. For this to work, the simulation directory must be prepared for running simulations (`icncsim makefile` must have been run before in the directory).

For batch simulation a `run` target is provided as well.

3.3.2 Creating Testcases using C

Creating Testcases using the C programming language follows the same procedure as described in Section 3.3.1, but using `riscv_core.tc_sanity` as a template.

The `main` function will be called by the default reset-handler. In order to write a custom interrupt handler, define the appropriate function out of those given in Section 3.2.2.

3.3.3 Disable Compressed Instructions

The default software setup for C code may generate compressed instructions. To disable this for small groups or to temporarily generate code without them you can change `ARCHFLAGS` in `Makefile.common` appropriately.

4 Task Extension for Larger Groups

'Don't interrupt me while I'm interrupting.'

(Winston Churchill)

4.1 Architecture

The SoC is to be extended by a programmable interrupt controller and the number of interrupts is increased to 8. The modified block diagram is shown in figure 4.1. Interrupt inputs are connected to the controller which forwards an interrupt id ($0 \ldots 7$) and an interrupt-enable signal towards the core. Incoming interrupts can be enabled and prioritized individually via the controller. The configuration is done via memory-mapped registers.

4.2 Interrupt Controller

The interrupt controller is a configurable module controlling the interrupt to be forwarded to the core.

Configuration is stored in memory mapped registers via a port to the memory controller. The memory controller will therefore map addresses `0x08000` to `0x0803f` to the local interrupt controller. The registers are shown in table 4.1.

For each of the 8 interrupts, the following registers can be configured or read:

- Enable, configured via bit n of the IRQ-Enable register for interrupt n. Only enabled interrupts are forwarded to the core, disabled interrupts are ignored.
- Mode, configured via bit n of the IRQ-Mode register. In mode 0, the interrupt input must be kept active until the interrupt is handled, in

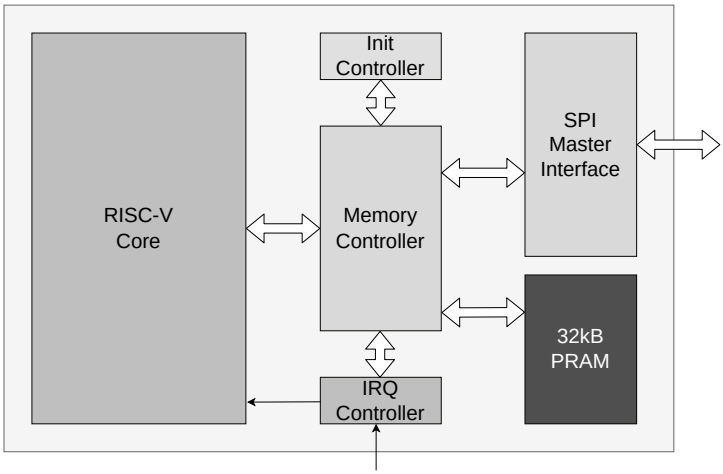


Figure 4.1: Kreacher Extended SoC Architecture

Table 4.1: Register map of IRQ controller

Address	Register	used bits	Comment
0x8000	IRQ-Enable	7:0	interrupt-enable vector
0x8004	IRQ-Mode	7:0	interrupt-mode vector
0x8008	IRQ-Status	7:0	interrupt status vector, read-only
0x8020	IRQ0-Prio	1:0	priority of interrupt 0
0x8024	IRQ1-Prio	1:0	priority of interrupt 1
0x8028	IRQ2-Prio	1:0	priority of interrupt 2
0x802c	IRQ3-Prio	1:0	priority of interrupt 3
0x8030	IRQ4-Prio	1:0	priority of interrupt 4
0x8034	IRQ5-Prio	1:0	priority of interrupt 5
0x8038	IRQ6-Prio	1:0	priority of interrupt 6
0x803c	IRQ7-Prio	1:0	priority of interrupt 7

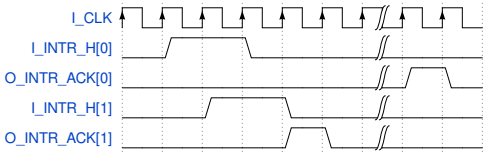


Figure 4.2: Interrupt request and acknowledge sequence in mode 1

mode 1 an active pulse of at least 2 clock-periods is sufficient for the interrupt to become active.

- Status, readable via bit n of the IRQ-Status register. A 1 indicates an active interrupt waiting to be handled (*pending*). This happens when more than one interrupt become active in a short time frame while only one of them can be handled.
- Priority, configured via IRQn-Prio register. If more than one interrupt are pending, the interrupt with lowest priority value is served first. If 2 interrupts are configured with the same priority, the one with lowest ID is served first.

The core indicates to the controller when it handles an interrupt so that the controller can clear pending interrupt states, trigger the appropriate interrupt-acknowledge output and forward the next interrupt to the core, if more interrupts are pending. Figure 4.2 shows an interrupt and acknowledge sequence with interrupt 0 and 1 in mode 1 and interrupt 1 having a lower priority value than 0.

After reset, all interrupts are disabled, in mode 0 and have a priority of 0.

4.3 Software

For the increased number of interrupts the handler addresses are modified as shown it table 4.2. For the given software setup the handlers must be modified appropriately in order to be able to use compiled software.

Note: For accessing the interrupt controller from software it is useful to define an appropriate C-structure mapped to the controller's address.

Table 4.2: Modified reset and handler addresses

handler	address
IRQ0	`0x0000`
IRQ1	`0x0004`
IRQ2	`0x0008`
IRQ3	`0x000c`
IRQ4	`0x0010`
IRQ5	`0x0014`
IRQ6	`0x0018`
IRQ7	`0x001c`
Reset	`0x0020`
Illegal instruction	`0x0024`

Bibliography

[1] Krste Asanović Andrew Waterman, ed. *The RISC-V Instruction Set Manual*. Vol. I: Unprivileged ISA. Document Version 20191213. Dec. 2019.

[2] Krste Asanović Andrew Waterman, ed. *The RISC-V Instruction Set Manual*. Vol. II: Privileged Architecture. Document Version 20190608. June 2019.